Viteezy — Project Handover Document

Document Version: 1.0

Date: June 17, 2026

Project: Viteezy Mobile Application (viteezy-v2)

Package Name: com.viteezy.app

App Version: 1.0.0+1

Table of Contents

1. Project Overview

Project Name

Viteezy (viteezy)

Purpose and Goals

Viteezy is a health and wellness e-commerce mobile application focused on vitamins, supplements, and personalized health products. The app enables users to:

- Browse and purchase health products

- Take health quizzes to receive personalized supplement recommendations
- Manage subscriptions and memberships
- Interact with an AI health chat assistant
- Track supplement reminders
- Manage family member accounts
- Complete checkout with payment integration

The application is designed as a consumer-facing retail platform with membership tiers, subscription management, and multi-language support for European markets.

Target Platforms

Platform	Status	Notes
Android	Primary	applicationId: com.viteezy.app
iOS	Primary	Bundle display name: Viteezy
Web	Scaffold present	web/ folder exists; not primary focus
Windows / macOS / Linux	Scaffold present	Desktop folders exist; not production targets

The app is locked to portrait orientation on mobile devices.

Key Features Summary

- User authentication (email/password, Google Sign-In, Apple Sign-In, Member ID login)
- Home landing page with video hero, categories, and featured products
- Product browsing, filtering, search, and product detail pages
- Shopping cart with coupon/discount support
- Checkout and order completion flow
- Order history and order details
- Wishlist management
- Subscription management (pause, cancel, product updates)
- Membership plans and payment via WebView
- AI health chat with session history
- Health quiz flow with personalized recommendations
- Push notifications (OneSignal) with deep-link routing
- Supplement reminders (create, toggle, history)
- Family member management
- Multi-language support (English, Dutch, German, French, Spanish)
- Help center / FAQ and customer support
- Profile management and address book
- Firebase Remote Config license gate for release builds

2. Technology Stack

Flutter Version

- Flutter: 3.44.1 (stable channel)
- Dart SDK: ^3.10.0 (project constraint); runtime Dart 3.12.1
- DevTools: 2.57.0

State Management

GetX (get: ^4.7.3) — used for:

- Reactive state (Rx, Obx)
- Dependency injection (Get.put, Get.lazyPut, Bindings)
- Navigation (GetMaterialApp, Get.toNamed, Get.offAllNamed)
- Internationalization (Get.tr, AppTranslations)

> Note: Bloc, Cubit, Provider, and Riverpod are not used in this project.

Key Packages and Libraries

Package	Version	Purpose
get	^4.7.3	State management, routing, DI, i18n
dio	^5.9.1	HTTP client for API calls
http	^1.6.0	Supplementary HTTP (limited use)
shared_preferences	^2.5.4	Local persistence (tokens, locale, user prefs)
firebase_core	^4.4.0	Firebase initialization
firebase_auth	^6.1.4	Social auth (Google, Apple)
firebase_remote_config	^6.1.4	License key gate for release builds
google_sign_in	^6.3.0	Google OAuth
sign_in_with_apple	^7.0.1	Apple Sign-In (iOS)
onesignal_flutter	^5.4.1	Push notifications
flutter_screenutil	^5.9.3	Responsive UI scaling (375×812 design)
flutter_svg	^2.2.3	SVG icon rendering
cached_network_image	^3.4.1	Image caching
shimmer	^3.0.0	Loading skeleton placeholders
lottie	^3.3.2	Lottie animations
video_player	^2.11.0	Home hero video playback
webview_flutter	^4.13.1	Payment and external content
image_picker	^1.2.1	Profile image selection
permission_handler	^12.0.1	Runtime permissions

pinput	^6.0.2	OTP input UI
flutter_widget_from_html	^0.17.1	HTML content rendering
url_launcher	^6.3.2	External links
share_plus	^12.0.1	Product sharing
intl	^0.20.2	Internationalization utilities
fluttertoast	^9.0.0	Toast messages
flutter_gen_runner	^5.12.0	Asset code generation
build_runner	^2.11.1	Code generation runner

3. Project Architecture

Architecture Pattern

The project follows a feature-based layered architecture with GetX MVC conventions:

```
Presentation Layer (Views + Controllers + Bindings)
        ↓
Repository Layer (API abstraction)
        ↓
Service Layer (ApiClient, Firebase, OneSignal, etc.)
        ↓
Models + Utils
```

This is not strict Clean Architecture. There is no separate domain/ layer with use cases. Business logic lives primarily in Controllers, with data access delegated to Repositories in lib/core/repositories/.

Folder Structure (High Level)

```
viteezy-v2/
├── lib/
│   ├── main.dart                # App entry point
│   ├── firebase_options.dart    # Firebase platform config
│   ├── core/                    # Shared infrastructure
│   ├── presentation/            # Feature modules (UI)
│   ├── generated/               # Generated i18n files
│   └── gen/                     # FlutterGen asset outputs
├── assets/                      # Fonts, icons, images, video, animations
├── android/                     # Android native project
├── ios/                         # iOS native project
├── web/                         # Web scaffold
├── test/                        # Tests
└── pubspec.yaml
```

Code Organization Strategy

18. Feature modules under lib/presentation/ — each feature has views/, controllers/, and bindings/.
19. Shared infrastructure under lib/core/ — repositories, models, services, widgets, theme, routes.

20. Bindings wire controllers to routes via GetX GetPage definitions in app_pages.dart.
21. Permanent singleton services registered in main.dart via initServices().
22. Asset references generated via flutter_gen into lib/gen/.

4. Folder Structure Breakdown

lib/

Root of all Dart application code. Contains main.dart, Firebase options, and subdirectories for core and presentation layers.

Subfolder	Purpose
core/	Shared application infrastructure
presentation/	Feature-specific UI and controllers
generated/	Auto-generated internationalization message files
gen/	Auto-generated asset references (assets.gen.dart, fonts.gen.dart)

lib/core/

Central shared module. Not a separate Flutter package — it is a logical grouping within the app.

Subfolder	Purpose
core/repositories/	Data access layer; one repository per domain (auth, cart, orders, etc.)
core/models/	JSON-serializable data models and API response types
core/services/	Singleton services (API client, Remote Config, OneSignal, cart count)
core/widgets/	Reusable UI components (buttons, loaders, app bar, drawer)
core/theme/	Colors, text styles, theme data, string constants
core/routes/	Route names (app_routes.dart) and page definitions (app_pages.dart)
core/utils/	Helpers (validators, preferences, constants, dialog service)
core/l10n/	Localization service and translation maps
core/exceptions/	AppException and ExceptionHandler for API errors

lib/presentation/

Feature-based UI modules.

Subfolder	Purpose
presentation/auth/	Login, signup, forgot/reset/change password

presentation/onboard/	Splash screen, app locked screen
presentation/main/	All post-onboarding features (dashboard, cart, etc.)

data/

Not Available as a top-level directory. Data access is implemented via lib/core/repositories/ and lib/core/models/.

domain/

Not Available as a top-level directory. No separate domain/use-case layer; controllers call repositories directly.

presentation/ (within features)

Each feature module typically contains:

```
feature_name/
├── bindings/       # GetX dependency injection
├── controllers/    # Business logic + reactive state
├── views/          # UI screens and widgets
└── widgets/        # Feature-specific widgets (optional)
```

widgets/

Shared widgets live in lib/core/widgets/. Feature-specific widgets live inside their respective feature folders (e.g., quiz_chat/widgets/).

services/

Located at lib/core/services/:

Service	Purpose
api_service.dart	Dio-based HTTP client (ApiClient)
api_endpoints.dart	Centralized API endpoint constants
remote_config_service.dart	Firebase Remote Config license validation
onesignal_notification_service.dart	Push notification setup and deep links
global_settings_service.dart	Landing page and general settings cache
cart_count_service.dart	Global cart badge count
pending_action_service.dart	Deferred actions after login
language_data_refresh_service.dart	Refresh data on locale change

5. Setup & Installation Guide

Prerequisites

Requirement	Version / Notes
Flutter SDK	3.44.x or compatible with Dart ^3.10.0
Dart SDK	^3.10.0
Android Studio / VS Code	With Flutter and Dart plugins

Xcode	Required for iOS builds (macOS only)
CocoaPods	Required for iOS dependencies
JDK	17 (Android compileOptions / kotlinOptions)
Firebase project	Configured (google-services.json, GoogleService-Info.plist)
OneSignal account	App ID configured in app_constant.dart

Install Dependencies

```
cd viteezy-v2
flutter pub get
```

Generate Assets and Code

```
dart run build_runner build
```

This generates:

- lib/gen/assets.gen.dart
- lib/gen/fonts.gen.dart
- Internationalization files under lib/generated/

Run the Project

Debug (development — license gate bypassed in debug mode):

```
flutter run
```

With license key for release-like testing:

```
flutter run --dart-define=APP_LICENSE_KEY=your_secret_key
```

Specific device:

```
flutter devices
flutter run -d <device_id>
```

Environment Configuration

The project does not use a .env file. Configuration is managed through:

Configuration	Location
API base URL (live/dev)	lib/core/utils/app_constant.dart — isLiveMode flag
Quiz API base URL	lib/core/utils/app_constant.dart — quizBaseUrl
OneSignal App ID	lib/core/utils/app_constant.dart
Google OAuth client IDs	lib/core/utils/app_constant.dart
Firebase options	lib/firebase_options.dart (generated by FlutterFire CLI)
Android Firebase	android/app/google-services.json
iOS Firebase	ios/Runner/GoogleService-Info.plist

Release license key	--dart-define=APP_LICENSE_KEY=... at build time
Remote Config license	Firebase Console → app_license_key parameter

Switching API environment:

Edit lib/core/utils/app_constant.dart:

```
bool isLiveMode = true;  // true = staging-v2.viteezy.com, false = dev server
```

6. Build & Deployment

Android

Debug APK:

```
flutter build apk --debug
```

Release APK:

```
flutter build apk --release --dart-define=APP_LICENSE_KEY=your_secret_key
```

App Bundle (Play Store):

```
flutter build appbundle --release --dart-define=APP_LICENSE_KEY=your_secret_key
```

Output locations:

- APK: build/app/outputs/flutter-apk/
- AAB: build/app/outputs/bundle/release/

Release signing: Currently uses debug signing in android/app/build.gradle.kts. Production releases require configuring a release keystore (JKS credentials are stored separately in Viteezy IMP files/jks creds/ — do not commit to git).

iOS

```
flutter build ios --release --dart-define=APP_LICENSE_KEY=your_secret_key
```

Then archive and distribute via Xcode. Requires valid provisioning profiles, Apple Push Notification certificate, and Sign in with Apple key.

Web

```
flutter build web --release --dart-define=APP_LICENSE_KEY=your_secret_key
```

Output: build/web/

Release Configuration Checklist

- [ ] Set isLiveMode = true in app_constant.dart

- [ ] Pass APP_LICENSE_KEY via --dart-define
- [ ] Configure Firebase Remote Config app_license_key to match
- [ ] Configure release signing (Android JKS, iOS certificates)
- [ ] Verify OneSignal, Google Sign-In, and Apple Sign-In credentials
- [ ] Bump version in pubspec.yaml (version: x.y.z+build)

7. State Management Explanation

Approach: GetX

State is managed using GetX Controllers with reactive variables (RxBool, RxInt, RxList, etc.) and Obx / GetBuilder widgets in views.

Dependency Injection Flow

23. Route-level bindings — Each GetPage in app_pages.dart specifies a binding that registers controllers.
24. Permanent services — Registered in main.dart via Get.put(..., permanent: true).
25. Lazy loading — Some bindings use Get.lazyPut for on-demand controller creation.

Example Flow: Login → API → UI

```
LoginScreen (View)
    ↓ user taps Login
LoginController.login()
    ↓ validates input
AuthRepository.login(body: {...})
    ↓
ApiClient.request(url, type: POST, body)
    ↓ attaches Bearer token if present
Dio HTTP POST → https://staging-v2.viteezy.com/api/v1/auth/login
    ↓
ApiResponse parsed
    ↓ onSuccess
PrefService.setValue(PrefKeys.accessToken, token)
LoginController.isLoading = false
Get.offAllNamed(AppRoutes.dashboard)
    ↓
DashboardView renders with HomeView tab
```

Global Reactive Services

Service	Reactive State	Usage
CartCountService	Cart item count	Bottom nav badge, drawer
GlobalSettingsService	Landing page data, settings	Home screen content
DashboardController	selectedBottomNav	Tab switching

Session Invalidation

When API returns "Invalid token", ApiClient automatically:

26. Clears SharedPreferences
27. Signs out Firebase and Google
28. Resets cart count
29. Navigates to dashboard home tab (guest mode)

8. API Integration

API Structure

All main APIs follow a standard response envelope:

```
{
  "success": true,
  "status": 200,
  "message": "Request successful.",
  "data": { ... },
  "pagination": { ... }
}
```

Parsed by ApiResponse in lib/core/models/api_response.dart.

Base URL Handling

Environment	Base URL	Toggle
Live (staging)	https://staging-v2.viteezy.com/api/v1/	isLiveMode = true
Development	http://167.71.225.133:8050/api/v1/	isLiveMode = false
Quiz service	http://139.59.32.181:8000/api/v1/	Separate quizBaseUrl

URL construction: BaseRepository.getFullUrl(endpoint, withLang: true) appends ?lang=en (or current locale).

Authentication Flow

30. Email/Password: POST auth/login → receive accessToken → stored in PrefKeys.accessToken
31. Google: Firebase Google Sign-In → POST auth/google/login with Firebase ID token
32. Apple: Apple credential → POST auth/apple-login
33. Member ID: Special login identifier validated client-side, sent to login API
34. Registration: POST auth/register → OTP verification via auth/verify-otp
35. All authenticated requests: Authorization: Bearer <accessToken> header added by ApiClient

Error Handling Strategy

Layer	Responsibility
ExceptionHandler	Maps Dio/HTTP errors to user-friendly messages
ApiClient	Parses responses, invokes onSuccess / onError callbacks
Controllers	Display errors via toast, dialog, or inline validation

Token expiry	Auto-logout and redirect to dashboard on "Invalid token"

HTTP status codes 400–504 are mapped to specific user messages in app_exception.dart.

9. Key Modules / Features

9.1 Authentication (presentation/auth/)

Item	Detail
Description	Login, signup, password recovery, OTP verification, change password
Main Files	login_controller.dart, signup_controller.dart, auth_repository.dart
Flow	Splash → Dashboard (guest) → Login screen on protected action → Token stored → Full access

9.2 Onboarding (presentation/onboard/)

Item	Detail
Description	Splash screen with animation; app license lock screen
Main Files	splash_controller.dart, app_locked_screen.dart, app_root.dart
Flow	App launch → Remote Config check → Splash (4.8s) → Dashboard

9.3 Dashboard (presentation/main/dashboard/)

Item	Detail
Description	Main shell with bottom navigation (Home, Browse, Cart, Reminders, Account)
Main Files	dashboard_view.dart, dashboard_controller.dart
Flow	IndexedStack preserves tab state; drawer for additional navigation

9.4 Home (presentation/main/home/)

Item	Detail
Description	Landing page with hero video, categories, featured products, animations
Main Files	home_view.dart, home_controller.dart, home_repository.dart
Flow	Loads landing page data from GlobalSettingsService → renders tabs and video

9.5 Browse & Shop All (presentation/main/browse/, shop_all_view/)

Item	Detail
Description	Category browsing, product listing, filters (health goal, ingredients, sort)
Main Files	browse_controller.dart, shop_all_controller.dart, product_repository.dart
Flow	Browse categories → Shop All with filters → Product detail

9.6 Products (presentation/main/products/)

Item	Detail
Description	Product detail page, reviews, add to cart, wishlist toggle
Main Files	product_detail_view.dart, products_controller.dart
Flow	Product ID from route args → fetch product → display → cart/wishlist actions

9.7 Cart & Checkout (presentation/main/cart/, checkout/)

Item	Detail
Description	Cart management, coupons, checkout summary, payment, order complete
Main Files	cart_controller.dart, checkout_controller.dart, cart_repository.dart, checkout_repository.dart
Flow	Add items → Review cart → Checkout → Payment WebView → Order complete

9.8 Orders (presentation/main/orders/)

Item	Detail
Description	Order history list and order detail with review capability
Main Files	orders_controller.dart, orders_screen.dart, order_repository.dart
Flow	My Orders → Order detail → Optional product review

9.9 Wishlist (presentation/main/wishlist/)

Item	Detail
Description	Saved/favorited products with pagination
Main Files	wishlist_controller.dart, wishlist_view.dart
Flow	Toggle wishlist from product → View in wishlist tab/screen

9.10 Subscriptions (presentation/main/subscription/)

Item	Detail
Description	Active subscriptions, pause/cancel, product changes, activity history
Main Files	subscription_controller.dart, subscription_repository.dart
Flow	Subscription list → Details → Manage products / pause / cancel

9.11 Membership (presentation/main/membership/)

Item	Detail
Description	Membership plans, purchase, payment tracking, cancellation
Main Files	membership_controller.dart, membership_view.dart, memberships_repository.dart
Flow	View plans → Buy membership → Payment WebView → Track status

9.12 AI Chat (presentation/main/ai_chat/)

Item	Detail
Description	AI health assistant chat with session history
Main Files	ai_chat_controller.dart, ai_chat_repository.dart
Flow	Login link for chat → Start session → Chat → View history

9.13 Quiz & Recommendations (presentation/main/quiz_chat/, recommendation/)

Item	Detail
Description	Health assessment quiz with dynamic questions; personalized supplement recommendations
Main Files	quiz_binding.dart, quiz_screen.dart, recommendation_screen.dart
Flow	Start quiz → Answer questions (single/nested/date types) → Recommendation screen

9.14 Reminders (presentation/main/reminder/)

Item	Detail
Description	Supplement reminder scheduling, toggle, and history
Main Files	reminder_controller.dart, reminder_repository.dart
Flow	Create reminder → Toggle on/off → View

	history

9.15 Family Module (presentation/main/familymodule/)

Item	Detail
Description	Family account info and sub-member management
Main Files	family_controllers.dart, family_repository.dart
Flow	View family info → Add/remove sub-members

9.16 Profile & Addresses (presentation/main/profile/, addresses/, add_address/)

Item	Detail
Description	User profile, edit profile, address book management
Main Files	profile_controller.dart, addresses_controller.dart, profile_repository.dart
Flow	Profile tab → Edit profile / Manage addresses

9.17 Notifications (presentation/main/notification/)

Item	Detail
Description	In-app notification list with read/unread status
Main Files	notification_controller.dart, notification_repository.dart
Flow	Notification list → Mark as read → Deep link to relevant screen

9.18 Help & Support (presentation/main/help_center/, help_details/, support/)

Item	Detail
Description	FAQ categories, help articles, support contact
Main Files	help_center_controller.dart, support_controller.dart
Flow	Help Center → Category → Article details / Support form

9.19 WebView (presentation/main/webview/)

Item	Detail
Description	Generic in-app browser for payments and external URLs
Main Files	webview_screen.dart, payment_webview.dart
Flow	Navigate with URL argument → Load in WebView → Return on completion

10. UI/UX Structure

Navigation Approach

GetX Named Routes (GetMaterialApp with getPages and initialRoute).

- Not using Navigator 2.0 or GoRouter
- Route constants in AppRoutes
- Page definitions with bindings in AppPages
- Default transition: Transition.cupertino (300ms)
- Bottom navigation via IndexedStack in DashboardView (preserves tab state)
- Side drawer via AppDrawer for secondary navigation

Theme Management

- Primary theme: AppTheme.lightTheme (Material 3)
- Dark theme: AppTheme.darkTheme defined but themeMode: ThemeMode.light is hardcoded
- Colors: lib/core/theme/app_colors.dart
- Typography: lib/core/theme/text_styles.dart with custom fonts
- Default font: saansTrial (Saans TRIAL family)

Responsive Design

- flutter_screenutil with design size 375 × 812 (iPhone X baseline)
- ScreenUtilInit wraps the main app shell in app_root.dart
- .w, .h, .sp extensions used throughout for scaling
- Portrait-only orientation enforced in main.dart

11. Assets & Resources

Images

- Location: assets/images/
- Referenced via FlutterGen: Assets.images.*

Icons

- Location: assets/icons/ (SVG format)
- Package: flutter_svg
- Referenced via: Assets.icons.*

Fonts

Family	Files	Weights
saansTrial	SaansTRIAL-*.ttf	300–700
poppins	Poppins-*.ttf	300–700

headline	HEADLINES-BOLD.OTF	700

Animations & Video

- Lottie: assets/animations/
- Video: assets/video/ (home hero and promotional content)

Localization

- Enabled: Yes (flutter_intl in pubspec.yaml)
- Supported locales: English (en), Dutch (nl), German (de), French (fr), Spanish (es)
- Translation files: lib/core/l10n/app_translations.dart (+ extra batch files)
- Generated: lib/generated/intl/messages_en.dart
- Usage: 'key_name'.tr via GetX translations
- Persistence: Locale saved in SharedPreferences (PrefKeys.locale)
- API integration: lang query parameter appended to API URLs

12. Testing & QA

Unit Tests

Not Available — No dedicated unit test files beyond the default widget test.

Widget Tests

- File: test/widget_test.dart
- Status: Outdated boilerplate (references counter app pattern, not aligned with current AppRoot)
- Coverage: Effectively 0% meaningful coverage

Integration Tests

Not Available

Known Test Coverage

Type	Coverage
Unit	0%
Widget	Minimal / broken boilerplate
Integration	0%

Recommended QA Areas

- Authentication flows (email, Google, Apple, Member ID)
- Cart and checkout with payment WebView
- Subscription pause/cancel/product update flows
- Push notification deep links (cold start and foreground)
- Multi-language switching and API lang parameter
- Remote Config license gate on release builds

- Offline / poor network error handling

13. Performance Considerations

Optimization Techniques Used

Technique	Implementation
Image caching	cached_network_image for network images
Loading placeholders	shimmer package for skeleton UI
Tab state preservation	IndexedStack in dashboard (avoids rebuild on tab switch)
Lazy controller init	GetX Bindings with Get.lazyPut
Video lifecycle	HomeVideoRouteObserver pauses video on navigation away
Asset code generation	FlutterGen for type-safe asset references
Global settings cache	GlobalSettingsService caches landing page data

Memory Management

- Controllers use onClose() for disposing TextEditingController, AnimationController, VideoPlayerController
- HomeController tracks _isDisposed flag to prevent post-dispose updates
- WidgetsBindingObserver removed in onClose() for lifecycle listeners

Best Practices Followed

- Centralized API client with timeout configuration (30s connect/receive)
- Repository pattern for data access separation
- Reusable widget library in core/widgets/
- Portrait lock reduces layout complexity

14. Known Issues / Limitations

Bugs / Incomplete Features

Issue	Location	Description
Outdated widget test	test/widget_test.dart	Counter app test does not match actual app
TODO: Navigate to cart	custom_toast.dart	Toast action navigation not implemented
TODO: Category navigation	app_drawer.dart	Drawer category links incomplete
TODO: Restart subscription API	subscription_controller.dart	Restart endpoint not wired
TODO: Navigate to shop	orders_controller.dart	Post-order navigation placeholder

Duplicate login route	app_pages.dart	AppRoutes.login defined twice in pages list
Debug credentials	login_controller.dart	Hardcoded test email/password in kDebugMode
Release signing	build.gradle.kts	Android release uses debug signing config
Folder naming	presentation/main/ member/	Directory names contain spaces (binding path issues risk)
Dashboard tab label mismatch	dashboard_view.dart	Tab index 3 labeled as Wishlist in comments but loads ReminderView

Pending Features

- Full test suite (unit, widget, integration)
- Dark theme activation in production
- Desktop/web production support
- Complete drawer category navigation

Technical Debt

- No domain/use-case layer (fat controllers)
- Mixed naming conventions (ShopALlBinding typo, canselMembership typo in endpoints)
- Configuration hardcoded in app_constant.dart instead of environment flavors
- http package included alongside dio (redundant)
- Some commented-out routes and dead code in app_pages.dart

15. Future Improvements

Suggested Enhancements

36. Environment flavors — Use --dart-define or flutter_flavorizr for dev/staging/prod instead of boolean flags
37. Comprehensive testing — Unit tests for repositories, widget tests for critical flows
38. CI/CD pipeline — Automated builds, linting, and test runs on GitLab/GitHub
39. Error monitoring — Integrate Sentry or Firebase Crashlytics
40. Offline support — Cache product catalog and cart for offline viewing
41. Dark mode — Enable ThemeMode.system or user preference toggle
42. Secure storage — Migrate tokens from SharedPreferences to flutter_secure_storage
43. Code cleanup — Fix folder names with spaces, remove duplicate routes, update widget test

Scalability Improvements

44. Extract core/ into a separate package for multi-app reuse
45. Introduce a domain layer with use cases for complex business logic

46. Implement pagination abstraction for all list screens
47. Add API response caching layer (e.g., dio_cache_interceptor)
48. Modularize features into self-contained modules with clear boundaries

16. Credentials & Access

> Security Notice: Never commit secrets to version control. The git status shows sensitive files staged under Viteezy IMP files/ — these should be removed from git and stored in a secure vault.

API Configuration (Masked)

Item	Value
Live API Base URL	https://staging-v2.viteezy.com/api/v1/
Dev API Base URL	http://167.71.225.133:8050/api/v1/
Quiz API Base URL	http://139.59.32.181:8000/api/v1/
OneSignal App ID	a11aa583-**--**-8a7fdec981b6
Google OAuth (Android)	26828450688-****.apps.googleusercontent.com
Google OAuth (iOS)	26828450688-****.apps.googleusercontent.com
Firebase	Configured via firebase_options.dart and platform JSON/plist files
App License Key	Set via --dart-define=APP_LICENSE_KEY=*** at build time
Remote Config Key	Firebase parameter: app_license_key

External Service Access

Service	Access Notes
Firebase Console	Project configured in firebase.json; manage Remote Config, Auth providers
OneSignal Dashboard	Push notification campaigns, device segments, App ID
Apple Developer	Push certificate (.cer), Sign in with Apple key (.p8) in Viteezy IMP files/
Google Play Console	App ID: com.viteezy.app; requires JKS for signing
App Store Connect	iOS distribution; requires provisioning profiles

Admin Access

Not Available — No in-app admin panel. Backend administration is handled separately via the Viteezy server/admin dashboard (not part of this Flutter codebase).

17. Handover Notes

Important Developer Notes

49. Guest-first navigation: The app always routes to the dashboard after splash. Users can browse without logging in; login is required for protected actions (checkout, profile, etc.).
50. License gate: Release builds require matching APP_LICENSE_KEY dart-define and Firebase Remote Config app_license_key. Debug builds bypass this check automatically.
51. Token handling: On invalid token, the app clears session and returns to dashboard home — not the login screen. This is intentional for guest browsing continuity.
52. Asset generation: After adding new assets, run dart run build_runner build to regenerate lib/gen/ files.
53. Language changes: When switching locale, LanguageDataRefreshService triggers data refresh. API calls should use withLang: true in repository URL builders.
54. OneSignal deep links: Notification payloads support appRoute and query params (orderId, productId, subscriptionId, membershipId, ticketId, quizSessionId, expertId). Cold-start notifications are deferred until after splash → dashboard.
55. Video on home: HomeVideoRouteObserver manages video pause/resume on route changes. Be careful when modifying navigation around the home screen.
56. Payment flows: Membership and checkout payments use in-app WebView. Test on real devices with valid payment credentials.

Things to Be Careful About

- Do not commit JKS files, .p8 keys, .cer certificates, or credential text files to git
- Android release builds currently use debug signing — must be updated before Play Store submission
- Hardcoded debug login in LoginController should not ship in production builds (guarded by kDebugMode but verify)
- Folder paths with spaces (member/ binding/) may cause issues on some CI systems — consider renaming
- isLiveMode is a compile-time constant — changing environments requires code edit and rebuild
- Quiz API uses a separate base URL from the main API — ensure both servers are reachable in target environment

Key Contacts & Documentation

- Flutter documentation: https://docs.flutter.dev/
- GetX documentation: https://pub.dev/packages/get
- Firebase Flutter setup: https://firebase.google.com/docs/flutter/setup
- OneSignal Flutter SDK: https://documentation.onesignal.com/docs/flutter-sdk-setup

End of Document

This handover document was generated from codebase analysis of the Viteezy v2 Flutter project. Review and update periodically as the application evolves.